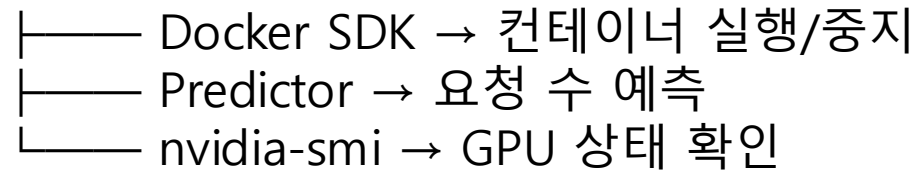


예상 사용자 이용 흐름도

[사용자 요청]



[FastAPI]



[GPU 컨테이너 할당]

호스트 준비물

✓ 호스트(너 컴퓨터)

해야 할 것:

1. FastAPI 서버 실행
2. Docker 설치 + 실행 상태 유지
3. GPU 드라이버 + NVIDIA Toolkit 설치

☐	Name	State	IP Address
☐	 s32210481	 ● Running	10.0.10.108

Showing 1-1 of 1 items < 1 > 20 / page ▾

저희 DKU 클라우드에는 GPU자원이 없는거 같습니다.
 하려면 개인 컴퓨터에서 해야할듯 합니다

```

master@DESKTOP-I67QF81: /r  ×  +  ▾
passwd: password updated successfully
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

master@DESKTOP-I67QF81:/mnt/c/Users/admin$ nvidia-smi
Mon Apr 27 21:40:40 2026

+---+
| NVIDIA-SMI 565.51.01                  Driver Version: 565.90          CUDA Version: 12.7
|
+---+
| GPU   Name                               Persistence-M | Bus-Id        Disp.A | Volatile Uncorr. E
CC | Fan  Temp  Perf    Pwr:Usage/Cap |      Memory-Usage | GPU-Util  Compute
M. |                                         |                      |                  MIG
M. |                                         |                      |                  |
+---+
| 0   NVIDIA GeForce RTX 4070 ...      On   | 00000000:01:00.0 Off |                  N
/A |                                         |                      |                  |
| N/A   37C    P8              3W / 30W |      0MiB / 8188MiB |      0%      Defau
lt |                                         |                      |                  N
/A |                                         |                      |                  |
+---+

+---+
| Processes:
| GPU   GI    CI          PID    Type    Process name                      GPU Memo
ry | ID    ID                                         |          Usage
|
+---+
| No running processes found
|
+---+

Windows 정품 인증
[설정]으로 이동하여 Windows를 정품 인증합니다.
master@DESKTOP-I67QF81:/mnt/c/Users/admin$

```

[사용자 요청]



[FastAPI]

- └── Docker SDK → 컨테이너 실행/중지
- └── Predictor → 요청 수 예측
- └── nvidia-smi → GPU 상태 확인



[GPU 컨테이너]



backend/

- └── main.py << 프론트로부터 들어온 요청관리 및 아래 파일들
여기서 실행
- └── docker_manager.py << GPU 할당
- └── gpu_monitor.py
- └── predictor.py
- └── logs.json

[사용자 요청]



HTML로 프론트 구현
실행누르면 FASTAPI랑 연동되게

```
<!DOCTYPE html>
<html lang="ko">
<head>
<meta charset="UTF-8">
<title>GPU Scheduler</title>
```

```
<style>
body {
  font-family: Arial;
  padding: 30px;
}
```

```
.section {
  margin-bottom: 20px;
}
```

```
button {
  padding: 8px 12px;
  margin: 5px;
  font-size: 16px;
  cursor: pointer;
}
```

```
#gpu {
  background: #111;
  color: #0f0;
  padding: 10px;
  white-space: pre-wrap;
}
```

```
#time {
  font-weight: bold;
}
</style>
</head>
```

```
<body>
```

```
<h2>GPU 모니터링</h2>
```

GPU 모니터링

서버 시간: -

GPU 상태:

조화 실패

요청 강도:

- 1 +

요청 보내기



1

docker_manager.py (GPU 컨테이너 실행)

<> Python



▶ 실행됨

```
import docker # Docker API 사용

client = docker.from_env() # 현재 Docker 엔진 연결

def run_container_async(gpu_id: int):
    """
    GPU를 할당해서 컨테이너 실행 (비동기)
    → 여러 요청 동시에 처리 가능 (MPS 활용)
    """

    container = client.containers.run(
        "my-ai-gpu", # 사용할 이미지 이름
        detach=True, # 백그라운드 실행

        # 🔥 GPU 할당 설정
        device_requests=[{
            "Driver": "nvidia",
            "DeviceIDs": [str(gpu_id)], # 사용할 GPU 번호
            "Capabilities": [["gpu"]]
        }]
    )

    return container.id # 컨테이너 ID 반환

def get_container_logs(container_id: str):
    """
    실행 완료된 컨테이너 로그 가져오기
    """

    container = client.containers.get(container_id)

    container.wait() # 작업 끝날 때까지 대기

    logs = container.logs().decode() # 결과 가져오기

    container.remove() # 컨테이너 삭제 (자원 ↓)

    return logs
```

backend/

|—— main.py

|—— docker_manager.py <<GPU할당

|—— gpu_monitor.py

|—— predictor.py

|—— logs.json

```
Python
import subprocess
import time
import docker

def get_monitor_data():
    """
    GPU 상태 + 서버 시간 반환
    """
    try:
        result = subprocess.check_output([
            "nvidia-smi",
            "--query-gpu=memory.used,memory.total,utilization.gpu",
            "--format=csv,noheader,nounits"
        ])

        lines = result.decode().strip().split("\n")

        gpus = []
        for i, line in enumerate(lines):
            used, total, util = map(int, line.split(","))

            gpus.append({
                "id": i,
                "used": used,
                "total": total,
                "util": util
            })

        return {
            "time": int(time.time()), # 서버 시간
            "gpus": gpus
        }

    except Exception as e:
        return {
            "time": int(time.time()),
            "error": str(e)
        }

def count_running_containers():
    """
    현재 실행 중인 컨테이너 개수
    → GPU 과부하 방지용
    """
    client = docker.from_env()
    containers = client.containers.list()
    return len(containers)
```

backend/

└── main.py

└── docker_manager.py

└── gpu_monitor.py <← 프론트 웹에서 표시할 값들
Gpu 점유율, 사용시간 등등

└── predictor.py

└── logs.json

backend/

|—— main.py << 프론트로부터 들어온 요청관리 및 아래 파일들
여기서 실행

|—— docker_manager.py
|—— gpu_monitor.py
|—— predictor.py
|—— logs.json

```
# -----  
# 사용자 요청 API  
# -----  
@app.post("/infer")  
def infer(data: dict, background_tasks: BackgroundTasks):  
    global model  
  
    load = data.get("load", 1) # 요청 강도  
  
    log_request() # 요청 기록  
  
    # 🔥 GPU 과부하 방지  
    if count_running_containers() >= MAX_CONTAINERS:  
        return {"error": "GPU 바쁨 (대기 필요)"}  
  
    # 🔥 load 만큼 컨테이너 실행  
    container_ids = []  
  
    for _ in range(load):  
        container_id = run_container_async(0) # GPU 0 사용  
        container_ids.append(container_id)  
  
        # 백그라운드에서 결과 처리  
        background_tasks.add_task(handle_container, container_id)  
  
    return {  
        "msg": "실행 시작",  
        "containers": container_ids  
    }  
  
# -----  
# 스케줄러 (AI 예측)  
# -----  
def scheduler_loop():  
    global model  
  
    while True:  
        try:  
            model = train_model() # 모델 학습  
  
            pred = predict_next(model) # 다음 요청 예측  
  
            print("예측:", pred)  
  
            # 🔥 pre-warming 가능  
            if pred > 3:  
                print("🔥 미리 GPU 준비 가능")  
  
        except Exception as e:  
            print("에러:", e)  
  
        time.sleep(10)  
  
# 서버 시작 시 실행  
threading.Thread(target=scheduler_loop, daemon=True).start()
```

스케줄러 AI 예측부분 코드는
gpu_monitor.py
에서 학습하고 예측한 값을 가
져오면 될거 같습니다.

backend/

- └── main.py
- └── docker_manager.py
- └── gpu_monitor.py
- └── predictor.py << AI 예측 모델
- └── logs.json

1. 요청 올 때마다 시간 기록
 2. 시간별 요청 개수로 변환
 3. 과거 5개 → 다음 1개 학습
 4. 최신 데이터로 다음 요청 수 예측
- 이거 기반으로 컨테이너별 할당할 리소스 정해주면 될듯 (ex 전체 GPU의 40%)
- 언제 실행? >> main.py에 gpu자원 요청 들어올때마다 실행

```
import json
import time
from collections import defaultdict
from sklearn.ensemble import RandomForestRegressor
LOG_FILE = "logs.json"

# 요청 기록 json 파일로 저장
def log_request() :
    log = { "timestamp": int(time.time()) }
    with open(LOG_FILE, "a") as f :
        f.write(json.dumps(log) + "\n")

# 시계열 생성(10초 단위) 시간을 10초 단위로 묶는다 >> 결과 [3, 1, 0, 5, 2 ...] >> 의미 첫 10초 → 3개 요청 다음 10초 → 1개 요청
def build_series() :
    counts = defaultdict(int)
    try :
        with open(LOG_FILE) as f :
            for line in f :
                log = json.loads(line)
                bucket = log["timestamp"] // 10
                counts[bucket] += 1
    except :
        return []
    return [counts[k] for k in sorted(counts)]

# 모델 학습 window = 5 << 과거 5개로 다음 요청 횟수를 맞춤
def train_model() :
    series = build_series()
    if len(series) < 10 :
        return None
    X, y = [], []
    window = 5
    for i in range(len(series) - window) :
        X.append(series[i:i + window])
        y.append(series[i + window])
    model = RandomForestRegressor()
    model.fit(X, y)
    return model

# 예측
def predict_next(model) :
    series = build_series()

    if not model or len(series) < 5 :
        return 0

    return model.predict([series[-5:]])[0]
```

협업을 어떻게 할까요?

[사용자 요청]



[FastAPI]

- └── Docker SDK → 컨테이너 실행/중지
- └── Predictor → 요청 수 예측
- └── nvidia-smi → GPU 상태 확인



[GPU 컨테이너]

파일 구조도?

HTML 코드

backend/

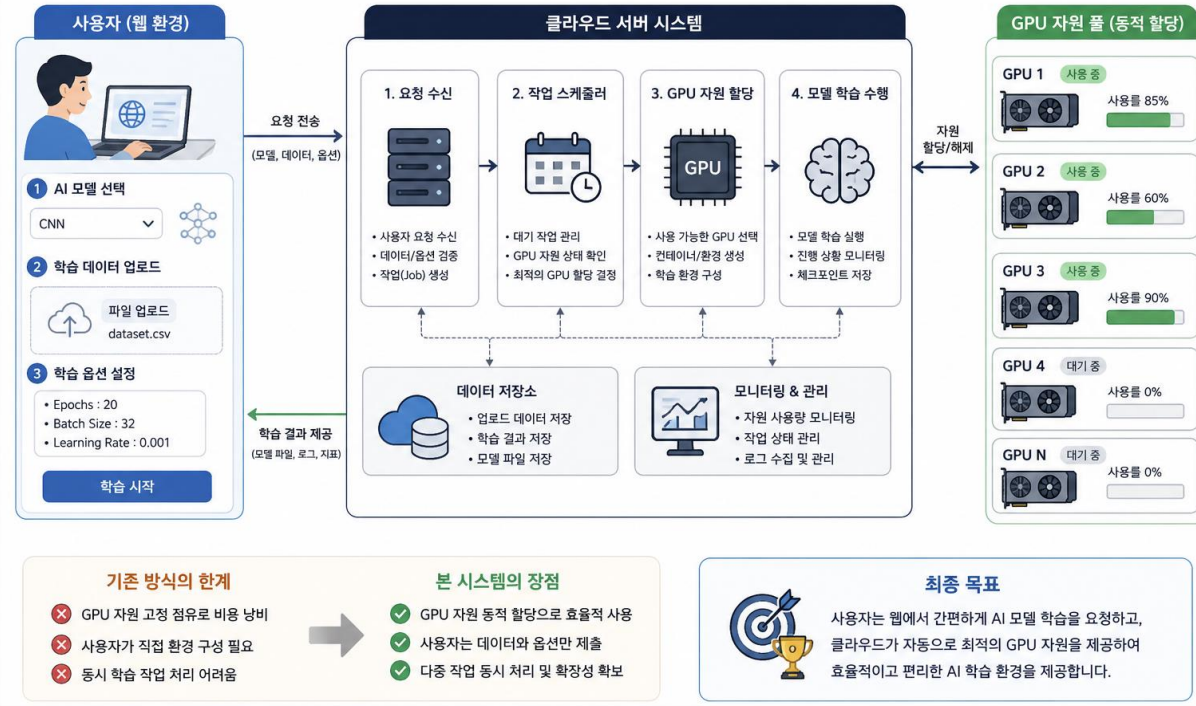
- └── main.py << 프론트로부터 들어온 요청관리 및 아래 파일들
여기서 실행
- └── docker_manager.py << GPU 할당
- └── gpu_monitor.py
- └── predictor.py
- └── logs.json

깃허브에 코드만 올려놓고 나머지 도커 설정이나 이런거는
readme에 작성해서 공유하는거 어떨까 싶습니다.(역할 맡으신 분이)
깃허브에서 코드 받고 파일배치하면 각자 컴퓨터 환경에서 실행 가능하도록 하게...

근데 저희가 GPU 자원 할당만하고 이걸로 뭐 사용하는 내용은 없어서
이 GPU 자원으로 뭐를 할까 생각하다가
다음 슬라이드에 그냥 제 의견을 적어봤습니다

AI 모델 학습을 위한 동적 GPU 자원 할당 시스템

사용자가 모델과 데이터를 업로드하면, 서버가 GPU 자원을 자동으로 할당하여 학습을 수행하고 결과를 제공합니다.



GPU 자원 활용과 AI 응용이라는 관점에서 고민한 해봤습니다.

사용자가 웹 환경에서 원하는 AI 모델(예: CNN 등)을 선택하고, 학습에 사용할 데이터 파일을 클라우드 환경으로 업로드하면, 서버가 GPU 자원을 동적으로 할당하여 해당 모델 학습을 수행하는 시스템을 구현하는 것입니다.

기존에는 GPU 자원이 고정적으로 점유되거나 사용자가 직접 학습 환경을 구성해야 하는 불편함이 있지만, 본 프로젝트에서는 사용자가 데이터와 학습 옵션만 제출하면 클라우드 환경에서 자동으로 GPU 자원을 분배하고 학습 작업을 처리하도록 하는 것을 목표로 합니다.

사용자가 현 상황 알 수 있게
대시보드로 알려주는 요소

1. 실행시간
2. gpu 사용률

클라이언트(출력)

클라이언트(입력)

사용자가 넘겨줘야할 거

- dataset 파일 업로드(학습시킬 파일)
- 모델 선택 (cnn or linear)
- epoch 설정 (학습 횟수)

FastAPI

1. 모델 파일 (weights)
2. 학습 로그 (loss, accuracy)
3. 설정값 (epoch, lr 등)

학습한 거
저장소

docker

GPU 스케줄러
(FIFO)

마운트

C
N
N

lin
ea
r

생성가능한 컨테이너들
Cnn는 이미지 학습 특화
Linear는 범용
(정말 간단한거)

예상 흐름

사용자 업로드



FastAPI



job queue(FIFO)



scheduler



docker gpu container



training



Dashboard로 진행상황 보기

각 컨테이너에 GPU 자원을 할당할때 얼마나 할당할지는
구현자가 생각을 해봐야 할거 같습니다

다음장은 코드 예시를 가져와봤습니다. 어쩌면 저희가 구현해야할 코드 전부 일 수도 있구요.(그리 길지 않습니다.)

1. 프론트 (업로드 화면)

```
<!DOCTYPE html>
<html>
<body>

<h2>GPU Training Scheduler</h2>

<form
  action="http://localhost:8000/submit_job"
  method="post"
  enctype="multipart/form-data"
>

  <label>Dataset:</label>
  <input type="file" name="dataset">

  <br><br>

  <label>Model:</label>
  <select name="model">
    <option value="cnn">CNN</option>
    <option value="mlp">MLP</option>
  </select>

  <br><br>

  <label>Epoch:</label>
  <input type="number"
    name="epochs"
    value="5">

  <br><br>

  <button type="submit">
    Start Training
  </button>

</form>

</body>
</html>
```

GPU Training Scheduler

Dataset: 선택된 파일 없음

Model:

Epoch:

2. FastAPI 백엔드 main.py

```
Python
from fastapi import FastAPI
from fastapi import UploadFile
from fastapi import File
from fastapi import Form

import os
import uuid

from scheduler import submit_job

app = FastAPI()

UPLOAD_DIR="uploads"

os.makedirs(UPLOAD_DIR,
            exist_ok=True)

@app.post("/submit_job")
async def submit_job_api(
    dataset: UploadFile=File(...),
    model: str=Form(...),
    epochs: int=Form(...)
):

    job_id=str(uuid.uuid4())

    path=f"{UPLOAD_DIR}/{job_id}.zip"

    with open(path,"wb") as f:
        f.write(
            await dataset.read()
        )

    submit_job(
        job_id,
        path,
        model,
        epochs
    )

    return {
        "job_id":job_id,
        "status":"queued"
    }
```

실행

```
Bash
uvicorn main:app --reload
```

3. 스케줄러

scheduler.py

Python



실행됨

```
import queue
import threading

from docker_controller import run_training

job_queue=queue.Queue()

gpu_busy=False

def submit_job(
    job_id,
    data_path,
    model,
    epochs
):
    job_queue.put(
        (job_id,
        data_path,
        model,
        epochs)
    )

def scheduler_loop():

    global gpu_busy

    while True:

        if (
            not gpu_busy
            and
            not job_queue.empty()
        ):

            job=job_queue.get()

            gpu_busy=True

            threading.Thread(
                target=execute_job,
                args=(job,)
            ).start()

def execute_job(job):

    global gpu_busy

    run_training(*job)

    gpu_busy=False
```

실행용:

Python



실행됨

```
from scheduler import scheduler_loop

scheduler_loop()
```



4. 학습 컨테이너

train.py

Python



▶ 실행됨

```
import os

model_type=os.getenv(
    "MODEL"
)

epochs=int(
    os.getenv(
        "EPOCHS"
    )
)

if model_type=="cnn":
    print("run cnn")
    train_cnn()

elif model_type=="mlp":
    print("run mlp")
    train_mlp()
```

4. cnn 학습코드 전체

CNN 예시

Python



▶ 실행됨

```
import torch
import torch.nn as nn

def train_cnn():

    device="cuda"

    model=nn.Sequential(
        nn.Linear(100,50),
        nn.ReLU(),
        nn.Linear(50,10)
    ).to(device)

    x=torch.randn(
        5000,100
    ).to(device)

    y=torch.randn(
        5000,10
    ).to(device)

    opt=torch.optim.Adam(
        model.parameters()
    )

    for e in range(epochs):

        pred=model(x)

        loss=(
            (pred-y)**2
        ).mean()

        opt.zero_grad()

        loss.backward()

        opt.step()

        print(loss.item())
```

6. 도커파일 설정, 출력 대시보드

Dockerfile

```
dockerfile
FROM pytorch/pytorch:latest
WORKDIR /app
COPY train.py .
CMD python train.py
```

빌드:

```
Bash
docker build -t training-job .
```

6. GPU 모니터링

monitor.py

```
Python
import subprocess

while True:
    subprocess.run([
        "nvidia-smi"
    ])
```

▶ 실행됨

또는

```
Bash
watch -n 1 nvidia-smi
```

역할분담

